

JavaScript Event Management

[Back to JS page](#)

Table of Contents

- [JavaScript Event Management](#)
 - [Adding Events](#)
 - [Example 1](#)
 - [Removing Events](#)
 - [Example 2](#)
 - [Blocking Events](#)
 - [Example 3](#)
 - [Document](#)
 - [Reference](#)

Adding Events

The `addEventListener()` method attaches an event handler to an element without overwriting existing event handlers.

You can add many event handlers to one element. You can also add many event handlers of the same type to one element (e.g., two "click" events):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Event Management</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Adding Events</h4>
<button id="myBtn">Click me</button>
<p id="demo"></p>

<script>
document.getElementById("myBtn").addEventListener("click", function() {
    document.getElementById("demo").innerHTML += "Button clicked!<br>";
});
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Removing Events

The `removeEventListener()` method removes event handlers that have been attached with the `addEventListener()` method.

To remove an event handler, the function used to add it must be a named (referenced) function:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Event Management</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Removing Events</h4>
<button id="myBtn">Click me</button>
<button id="removeBtn">Remove event</button>
<p id="demo"></p>

<script>
function myFunction() {
    document.getElementById("demo").innerHTML = "Button was clicked!";
}

document.getElementById("myBtn").addEventListener("click", myFunction);

document.getElementById("removeBtn").addEventListener("click", function() {
    document.getElementById("myBtn").removeEventListener("click", myFunction);
    document.getElementById("demo").innerHTML = "Event handler removed!";
});
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Blocking Events

The `preventDefault()` method cancels the event if it is cancelable, meaning that the default action that belongs to the event will not occur.

For example, this can prevent a link from opening the URL, or prevent a form from submitting:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Event Management</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Blocking Events</h4>
<a id="myLink" href="https://www.w3schools.com">Go to W3Schools</a>
<p id="demo"></p>

<script>
document.getElementById("myLink").addEventListener("click", function(event) {
    event.preventDefault();
    document.getElementById("demo").innerHTML = "Link click was prevented!";
});
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Document

Document in project

You can [Download PDF](#) file.

Reference

- [W3Schools JavaScript Event Management](#)